Task 1: Student Age Validation

Requirements

- Create a **Student** class.
- Create a private attribute `__age`.
- Use **@property** to read the age.
- Use **@age.setter** to update the age.
- Age must be between 18 and 60.
- If the age is invalid, display "Invalid age".

Concepts

__private, @property, @setter, data validation.

Program

```
# Task 1: Student Age Validation
class Student:
    def __init__(self, name, age):
        self.name = name
        self.__age = 18          # default (valid) value
        self.age = age           # goes through the setter for validation

    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, value):
        if 18 <= value <= 60:
            self.__age = value
        else:
            print("Invalid age")


s = Student("Alekhya", 22)
print("Age:", s.age)

s.age = 25               # valid update
print("Age after update:", s.age)

s.age = 10               # invalid (below 18)
s.age = 75               # invalid (above 60)
print("Age after invalid updates:", s.age)
```

Output

```
Age: 22
Age after update: 25
Invalid age
Invalid age
Age after invalid updates: 25
```

Explanation

The age is stored in the private attribute `__age`, so it cannot be changed directly from outside the class. The **@property** getter lets us read it, and the **@age.setter** checks that the value is between 18 and 60 before saving it. Invalid values print "Invalid age" and the old age stays unchanged.

Task 2: Bank Account

Requirements

- Create a **BankAccount** class.
- Create a private attribute `__balance`.
- Use **@property** to get the balance.
- Use **@balance.setter** to update the balance.
- Balance should not be negative.
- Create a *deposit()* method.
- Create a *withdraw()* method.
- Withdrawal amount should not be greater than the available balance.

Concepts

Encapsulation, private attributes, getter, setter, validation.

Program

```
# Task 2: Bank Account
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = 0
        self.balance = balance   # uses the setter

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount < 0:
            print("Balance cannot be negative")
        else:
            self.__balance = amount

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be greater than 0")
        else:
            self.__balance += amount
            print(f"Deposited {amount}. New balance: {self.__balance}")

    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal amount must be greater than 0")
        elif amount > self.__balance:
            print("Insufficient balance")
        else:
            self.__balance -= amount
            print(f"Withdrew {amount}. New balance: {self.__balance}")


acc = BankAccount("Alekhya", 5000)
print("Balance:", acc.balance)

acc.deposit(2000)
acc.withdraw(1500)
acc.withdraw(10000)      # more than available balance

acc.balance = -500       # negative balance not allowed
print("Final balance:", acc.balance)
```

Output

```
Balance: 5000
Deposited 2000. New balance: 7000
Withdrew 1500. New balance: 5500
Insufficient balance
Balance cannot be negative
Final balance: 5500
```

Explanation

The balance is hidden in `__balance` and can only change through the setter, *deposit()* and *withdraw()*. The setter rejects negative values, and *withdraw()* refuses to take out more than the available balance, so the account can never go into an invalid state.

Task 3: Employee Salary

Requirements

- Create an **Employee** class.
- Create a public attribute `name`.
- Create a protected attribute `_department`.
- Create a private attribute `__salary`.
- Use **@property** and **@salary.setter** for salary.
- Salary should not be less than 15,000.
- Create a method to display employee details.

Concepts

Public, protected, private, property, setter.

Program

```
# Task 3: Employee Salary
class Employee:
    def __init__(self, name, department, salary):
        self.name = name                # public
        self._department = department   # protected
        self.__salary = 15000           # private (default minimum)
        self.salary = salary            # goes through the setter

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, value):
        if value < 15000:
            print("Salary cannot be less than 15,000")
        else:
            self.__salary = value

    def display_details(self):
        print("Name      :", self.name)
        print("Department:", self._department)
        print("Salary    :", self.__salary)


emp = Employee("Ravi", "Engineering", 30000)
emp.display_details()

print("\nUpdating salary to 45000")
emp.salary = 45000
emp.display_details()

print("\nUpdating salary to 10000")
emp.salary = 10000

print("\nPublic    :", emp.name)
print("Protected :", emp._department)           # accessible, but only by convention
print("Private   :", emp.salary)                # accessed through the property
```

Output

```
Name      : Ravi
Department: Engineering
Salary    : 30000

Updating salary to 45000
Name      : Ravi
Department: Engineering
Salary    : 45000

Updating salary to 10000
Salary cannot be less than 15,000

Public    : Ravi
Protected : Engineering
Private   : 45000
```

Explanation

Public (*name*) can be accessed freely. **Protected** (*_department*) is meant for use inside the class and its subclasses; the single underscore is only a convention. **Private** (*__salary*) is name-mangled by Python and is accessed only through the property. The setter makes sure the salary is never below 15,000.

Task 4: Product Price

Requirements

- Create a **Product** class.
- Create a private attribute `__price`.
- Use **@property** to get the price.
- Use **@price.setter** to update the price.
- Price must be greater than 0.
- If an invalid price is given, display "Invalid price".
- Create a method to display product details.

Concepts

Private attributes, property, setter, validation.

Program

```
# Task 4: Product Price
class Product:
    def __init__(self, name, price):
        self.name = name
        self.__price = 1         # default (valid) value
        self.price = price       # goes through the setter

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, value):
        if value > 0:
            self.__price = value
        else:
            print("Invalid price")

    def display_details(self):
        print("Product:", self.name)
        print("Price  :", self.__price)


p = Product("Laptop", 55000)
p.display_details()

print("\nUpdating price to 52000")
p.price = 52000
p.display_details()

print("\nUpdating price to -100")
p.price = -100
p.display_details()
```

Output

```
Product: Laptop
Price  : 55000

Updating price to 52000
Product: Laptop
Price  : 52000

Updating price to -100
Invalid price
Product: Laptop
Price  : 52000
```

Explanation

The price is private and is changed only through the setter, which accepts values greater than 0. When an invalid price such as -100 is given, "Invalid price" is displayed and the previous price (52000) is kept.

Task 5: User Login System

Requirements

- Create a **User** class.
- Create a public attribute `username`.
- Create a private attribute `__password`.
- Use **@property** to access the password.
- Use **@password.setter** to change the password.
- Password must contain at least 8 characters.
- Create a *login()* method to verify the username and password.
- Display an appropriate message for successful and failed login.

Concepts

Encapsulation, private attributes, property, setter, validation, methods.

Program

```
# Task 5: User Login System
class User:
    def __init__(self, username, password):
        self.username = username
        self.__password = ""
        self.password = password        # goes through the setter

    @property
    def password(self):
        return self.__password

    @password.setter
    def password(self, value):
        if len(value) >= 8:
            self.__password = value
        else:
            print("Password must contain at least 8 characters")

    def login(self, username, password):
        if username == self.username and password == self.__password:
            print("Login successful. Welcome,", username)
        else:
            print("Login failed. Invalid username or password")


u = User("alekhya", "python123")

print("--- Login attempts ---")
u.login("alekhya", "python123")    # correct
u.login("alekhya", "wrongpass")    # wrong password
u.login("someone", "python123")    # wrong username

print("\n--- Changing password ---")
u.password = "abc"                 # too short
u.password = "newpass2026"         # valid
u.login("alekhya", "python123")    # old password now fails
u.login("alekhya", "newpass2026")  # new password works
```

Output

```
--- Login attempts ---
Login successful. Welcome, alekhya
Login failed. Invalid username or password
Login failed. Invalid username or password

--- Changing password ---
Password must contain at least 8 characters
Login failed. Invalid username or password
Login successful. Welcome, alekhya
```

Explanation

The password is stored in the private attribute `__password`. The setter accepts only passwords with at least 8 characters, and *login()* compares both the username and the password. After the password is changed, the old password no longer works. (In real applications passwords are stored as hashes, not plain text.)